HushWave

Single-channel speech noise reduction
built from first principles in the STFT domain

DSP mini project 6

Third-year B.Tech, Electronics and Communication Engineering
National Institute of Technology Goa

Group

Roll No. 20 Kartikey Tiwari
Roll No. 21
Roll No. 22

Repository: `github.com/kartikeyy12/hushwave-speech-denoiser`
Code revision 591a908, results generated on 2026-11-04
Every number in this report is produced by code and read from `results/results.json`.

Abstract

HushWave reduces stationary and slowly varying background noise in single-channel speech. The signal path is a 16 kHz, 32 ms short-time Fourier transform with weighted overlap-add, a noise power tracker (minima-controlled recursive averaging with a rise-rate limit), a decision-directed Wiener gain, and a zero-phase spectral high-pass. The radix-2 FFT, the windows, the STFT and overlap-add, the biquad, the trackers, the gain rules, the metrics, and the Welch estimator are all implemented in the project; libraries are used only for file input and output, plotting, special functions, and clearly marked external references.

The system was tuned on a development set and evaluated once on a held-out test set of 24 LibriSpeech utterances mixed with 11 noise conditions at 5 input SNRs (1320 cases). Its mean SNR improvement is 5.44 dB (95% interval 4.81 to 6.01 dB), against 5.20 dB for tuned spectral subtraction, 4.74 dB for a Wiener gain without decision-directed estimation, 0.52 dB for `scipy.signal.wiener`, and -1.93 dB for a general-purpose library denoiser. The margin over spectral subtraction is small but consistent (median 0.26 dB, Holm-corrected $p < 0.001$). Perceptual quality (PESQ-WB) is equal to spectral subtraction, and no gain-based method improves STOI. On two real recordings without a clean reference the noise reduction in pauses is only 2.8 dB and 1.9 dB, no better than spectral subtraction. The streaming engine has a fixed algorithmic delay of 31.9 ms and runs at 0.020 of real time on the measurement machine. A browser-based demonstration offers a replay mode and a live microphone mode.

Contents

1 Introduction **3**

1.1 Problem and scope . . . 3

1.2 What was built . . . 3

1.3 Rules the project follows . . . 3

2 Signal model and sampling **4**

2.1 Model . . . 4

2.2 Sampling frequency . . . 4

2.3 Quantisation . . . 4

3 Method **4**

3.1 Framing, windows, and overlap-add . . . 5

3.2 Pre-filter . . . 5

3.3 Noise power tracking . . . 5

3.4 Gain rules . . . 6

3.5 Streaming implementation . . . 7

4 Experimental setup **7**

4.1 Data . . . 7

4.2 Systems . . . 7

4.3 Tuning protocol . . . 8

4.4 Metrics . . . 8

4.5 Statistics 9
4.6 Real recordings 9

5 Results **9**
5.1 Overall comparison 9
5.2 Dependence on input SNR and noise type 10
5.3 Ablations 12
5.4 Time-domain and frequency-domain analysis 13
5.5 Real recordings 15
5.6 Own-voice corpus 15
5.7 Consistency between development and test 16
5.8 Latency and real-time performance 16

6 Demonstration **17**

7 Discussion and limitations **17**

8 Conclusion **18**

A Reproducing the results **19**

B Requirement traceability **20**

1 Introduction

1.1 Problem and scope

A microphone picks up speech together with fans, traffic, ventilation, and other people. When only one microphone is available there is no spatial information to exploit, so the separation has to rely on the statistics of the two signals: speech is non-stationary, sparse in time and frequency, and harmonic; many environmental noises are close to stationary over a second or more. The classical answer is short-time spectral attenuation. The signal is cut into overlapping frames, a noise power spectrum is tracked, and each time-frequency cell is multiplied by a gain that depends on how far its power rises above the noise.

The project brief, set by the course instructors, asks for an implementation of at least one DSP algorithm by the students themselves (not only library calls), the mathematical formulation, time-domain and frequency-domain analysis, raw against processed signals with numerical measures, a justified sampling frequency, and a working real-time or near-real-time demonstration. Appendix B maps each requirement to the place in this report and in the repository where it is met.

1.2 What was built

- A complete enhancement engine in `src/hushwave/`: radix-2 FFT, windows, STFT with weighted overlap-add, biquad filter, two noise trackers, four gain rules, and the evaluation metrics, each implemented from its definition.
- A streaming implementation with a fixed delay, shown to produce the same samples as the batch implementation (largest difference 0).
- A reproducible experiment with separate development and test data, utterance-level confidence intervals, and corrected significance tests.
- A demonstration interface with a replay mode and a live microphone mode.

1.3 Rules the project follows

Three rules shape everything below.

1. **From scratch.** All algorithms of the signal path and the metrics are implemented in the project. External code is limited to reading and writing files, plotting, special functions, and the reference implementations used for comparison and for two perceptual metrics (`pesq`, `pystoi`, `noisereduce`, `scipy.signal.wiener`). No learned model is used.
2. **No typed numbers.** Every number in this report, and every table, is generated by code from `results/results.json`. Before a report is built, a script checks the qualitative statements of the text against the data and stops if one no longer holds.
3. **One test evaluation.** Parameters were chosen on development data only. The test set was evaluated once (1 recorded evaluation, tuning file hash prefix 5faa0613f52ab9d5). A second evaluation is refused by the code unless forced, and would have to be disclosed.

2 Signal model and sampling

2.1 Model

The microphone signal is modelled as the sum of speech $s[n]$ and additive noise $d[n]$:

$$y[n] = s[n] + d[n]. \tag{1}$$

Speech and noise are assumed uncorrelated. The enhancement goal is an estimate $\hat{s}[n]$ with less noise and as little change to the speech as possible. The input signal-to-noise ratio of a mixture is

$$\mathrm{SNR}_{\mathrm{in}} = 10 \log_{10} \frac{\sum_n s^2[n]}{\sum_n d^2[n]}. \tag{2}$$

Test mixtures are built by scaling the noise by $g = \sqrt{\overline{s^2} / (\overline{d^2}\, 10^{\mathrm{SNR}/10})}$ so that this ratio equals the target.

2.2 Sampling frequency

Speech carries intelligibility information up to about 8 kHz, and a sampling rate f_s can represent content only below $f_s/2$. The project therefore uses $f_s = 16$ kHz. This keeps the fricatives that matter for intelligibility, matches the wide-band PESQ and STOI evaluation tools and the LibriSpeech and DEMAND corpora, and keeps the cost of the engine low. A higher rate would spend most of the computation on bins that hold no speech; a lower rate (8 kHz) would remove the 4 to 8 kHz range.

Resampling a recording that was captured at a higher rate needs an anti-alias filter. A direct demonstration in the repository decimates a two-tone signal from 48 kHz to 16 kHz. The 11 kHz tone folds to 5000 Hz. Without a filter the alias has a level of 0.0 dB relative to the in-band tone; with a linear-phase FIR low-pass it is -68.1 dB.

2.3 Quantisation

For a full-scale sine, an N-bit uniform quantiser has SQNR $\approx 6.02N + 1.76$ dB. Speech sits well below full scale, so the usable figure is lower. Table 1 lists the SQNR for signals at several levels below full scale. At 16 bits, a signal 40 dB below full scale still has 58.1 dB, far below the noise levels of any real room; at 8 bits the same signal would have only 9.9 dB. The engine works in 64-bit floating point internally and files are stored as 16-bit PCM.

Level below full scale (dB)	8 bit	12 bit	16 bit
0	49.9	74.0	98.1
20	29.9	54.0	78.1
40	9.9	34.0	58.1
60	−10.1	14.0	38.1

Table 1. SQNR of a uniform quantiser for a sine at the given level below full scale.

3 Method

Figure 1 shows the signal path. The next sections describe each block.

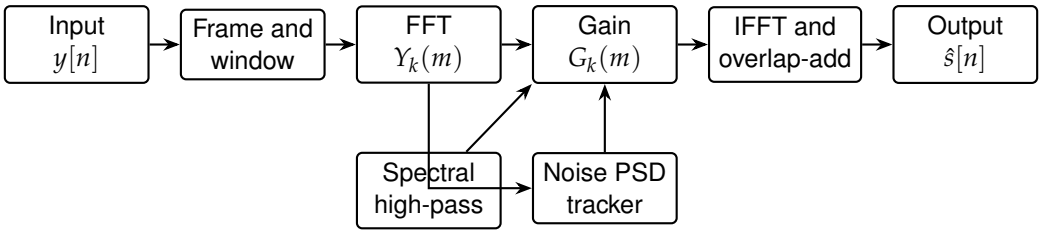


Figure 1. Signal path. The noise tracker and the high-pass act on the spectrum; the gain is applied to the noisy coefficient and the result is overlap-added.

3.1 Framing, windows, and overlap-add

The input is cut into frames of $N = 512$ samples (32 ms) with a hop of $H = 256$ samples (16 ms, 50% overlap). Each frame is multiplied by a square-root Hann window $w[n]$ and transformed with a radix-2 FFT of length N, giving $N/2 + 1 = 257$ bins spaced $f_s/N = 31.25$ Hz apart:

$$Y_k(m) = \sum_{n=0}^{N-1} w[n]\, y[mH + n]\, e^{-j2\pi kn/N}. \tag{3}$$

After the gain is applied, each frame is transformed back, multiplied by the same window, and added to the output at its position. With a square-root Hann window and 50% overlap, w^2 sums to one, so an unmodified signal is reconstructed exactly. A frame length of 32 ms is the usual compromise: short enough that speech is close to stationary within a frame, long enough to resolve the harmonics of a voice. The ablation in Section 5.3 confirms it against 16 ms and 64 ms frames.

3.2 Pre-filter

Energy below about 80 Hz is rumble and handling noise and carries no speech information. Two implementations of an 80 Hz second-order Butterworth high-pass are available.

- A *causal biquad* in direct form. It adds a frequency-dependent phase shift of tens of degrees at 100 to 300 Hz. When the result is compared with an unfiltered clean reference, this phase shift counts as error.
- A *zero-phase spectral mask* with the same magnitude response, applied as a real per-bin gain. A real, positive gain has no phase, so the speech is not shifted. The development data selected this variant, and the ablation confirms it (Section 5.3).

The magnitude of the 80 Hz second-order Butterworth high-pass, sampled at the bin centres, is

$$|H(f)| = \left[1 + \left(\tan(\pi f_c/f_s)/\tan(\pi f/f_s)\right)^4\right]^{-1/2}. \tag{4}$$

3.3 Noise power tracking

All gain rules need the noise power spectrum $\lambda_d(k, m)$. It cannot be measured directly while speech is present, so it is tracked from the noisy signal. Two trackers are implemented.

Minimum statistics. The periodogram $|Y_k(m)|^2$ is smoothed recursively over time, and the smoothed value is minimised over a sliding window of up to 96 frames (about 1.5 s), kept as six sub-windows of 16 frames. Because the minimum of a random signal is biased low, the result

is multiplied by a bias factor computed for stationary Gaussian noise. The assumption is that within 1.5 s the signal falls to the noise floor at least once in every bin.

Minima-controlled recursive averaging (MCRA). The smoothed periodogram $S(k,m)$ is divided by its running minimum $S_{\min}(k,m)$. If the ratio exceeds a threshold δ, speech is declared present in that cell. The presence indicator is smoothed into a probability $p(k,m)$, and the noise estimate is updated with a rate that is slow when speech is likely and fast when it is not:

$$\lambda_d(k,m) = \alpha_d(k,m)\,\lambda_d(k,m-1) + \big(1-\alpha_d(k,m)\big)\,|Y_k(m)|^2, \tag{5}$$

with $\alpha_d = \alpha_d^0 + (1-\alpha_d^0)\,p(k,m)$. The threshold selected on development data is $\delta = 3$.

Rise-rate limit. A minimum tracker jumps upward when the last pause leaves its window during continuous speech, and the noise estimate then follows the speech. The limit lets the estimate fall at once but rise by at most 3 dB per second. The limit is optional and was selected on development data; Section 5.3 shows its effect on the test data.

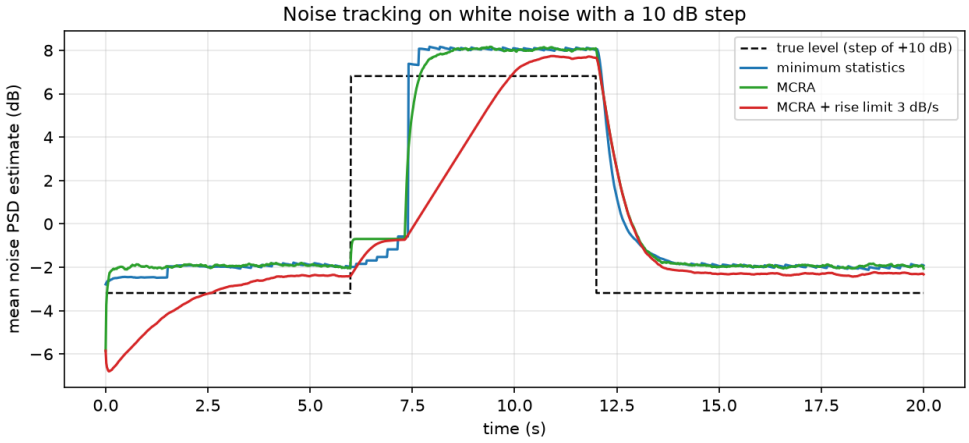


Figure 2. Tracking of white noise with a 10 dB step up and down. The rise limit makes the estimate follow an increase slowly, which protects speech and delays recovery after a genuine noise increase.

3.4 Gain rules

Let $\gamma_k(m) = |Y_k(m)|^2/\lambda_d(k,m)$ be the a posteriori SNR and $\xi_k(m)$ the a priori SNR. Every rule returns a gain G with $g_{\min} \le G \le 1$ and the enhanced coefficient is $G\,Y$.

Magnitude spectral subtraction (B1). $G = \max\big(1 - a/\sqrt{\gamma},\; g_{\min}\big)$ with over-subtraction factor a. The tuned value is $a = 1$ and the floor is $g_{\min} = 0.05$.

Wiener gain. $G = \xi/(1+\xi)$. The a priori SNR is estimated with the decision-directed approach of Ephraim and Malah:

$$\hat{\xi}_k(m) = \alpha\,\frac{\hat{A}_k^2(m-1)}{\lambda_d(k,m-1)} + (1-\alpha)\,\max\big(\gamma_k(m)-1,\,0\big), \tag{6}$$

where $\hat{A}_k(m-1)$ is the enhanced amplitude of the previous frame. A value of α close to one smooths ξ strongly and suppresses isolated bursts of residual noise. The tuned value is $\alpha = 0.9$. The recursion starts from zero, so the first frames are attenuated strongly rather than passed through unattenuated.

Log-spectral amplitude gain. $G = \frac{\xi}{1+\xi} \exp\big(\frac{1}{2} E_1(v)\big)$ with $v = \xi\gamma/(1+\xi)$ and E_1 the exponential integral. It is implemented and was part of the structure search on development data; the search did not select it.

Plain-SNR Wiener gain (B4). The same Wiener gain with the instantaneous estimate $\xi = \max(\gamma - 1, 0)$. It isolates the contribution of decision-directed estimation.

An optional recursive smoother $G \leftarrow \mu G_{\text{prev}} + (1-\mu)G$ is available. The tuned value is $\mu = 0$, that is, none.

3.5 Streaming implementation

The streaming engine accepts blocks of any length. It keeps the unfinished frame, the overlap-add tail, the biquad state, and the trackers between calls. Because a sample is final only after the last frame that contains it has been processed, the output is delayed by $N - 1 = 511$ samples, that is 31.9 ms at 16 kHz. This delay is constant and independent of the block size. The streamed output equals the batch output to within 0 in the largest absolute sample difference, which is checked by the test suite for the tuned configuration and for random block sizes.

4 Experimental setup

4.1 Data

Speech. LibriSpeech *dev-clean* is the primary corpus. Development and test utterances come from disjoint speakers (the split manifest is part of the repository and its validity is checked in the verification step). The test set has 24 utterances with a mean speech activity of 83% of blocks. The own-voice recording is a secondary corpus: the first half of the recording provides development segments and the second half provides 6 test segments, with no shared samples.

Noise. There are 11 noise conditions: six synthetic generators (white, pink, mains hum, babble from four speech sources, amplitude-modulated noise, and sparse transients), three DEMAND categories (hallway, cafeteria, street traffic), and the fan and corridor noise recorded by the group. Recordings and DEMAND files are split in time: the first half serves development, the second half the test.

Cases. The test set crosses 24 utterances, 11 noise conditions, and input SNRs of -5, 0, 5, 10, 15 dB, which gives 1320 cases. Each case has its own seed derived from its identity, so results do not depend on the number of worker processes. All systems start cold: there is no noise-only lead-in.

4.2 Systems

B0 Unprocessed input.

B1 Spectral subtraction on the same pipeline, with its own tuned parameters.

B2 `scipy.signal.wiener`, an external adaptive filter, with a window tuned on development data.

B3 `noisereduce` with default settings, an external general-purpose denoiser.

B4 Wiener gain with the plain a priori SNR, on the same pipeline.

B5 HushWave: Wiener gain with decision-directed SNR, MCRA tracking with $\delta = 3$, rise limit 3 dB/s, spectral high-pass, $g_{\min} = 0.03$, no gain smoothing, 32 ms frames.

B1 and B4 share every component of B5 except the one that defines them, so differences between them are attributable to that component. B2 and B3 show how generic tools behave on the same data.

4.3 Tuning protocol

Tuning used the 220 development cases only, with the objective $J = \frac{1}{2}(\Delta\text{SNR} + \Delta\text{segSNR})$ averaged over cases. The search had three stages: 24 structures (rule, tracker, pre-filter), a factorial grid over $(g_{\min}, \mu, \alpha)$, and sweeps of the rise limit, the tracker parameter, and the alternative rule. A first version that varied one parameter at a time missed an interaction between $g_{\min}$ and μ; this was found in a rehearsal on development data and the grid was made factorial before the real run. B1 and B4 were tuned on the same development cases, so that the comparison does not favour B5. Table 2 lists the development scores.

System	DEV objective J (dB)
B1 spectral subtraction (G1)	4.65
B2 scipy.signal.wiener	−0.29
B3 noisereduce (default)	−2.48
B4 Wiener, plain SNR	4.40
B5 HushWave (full)	5.06

Table 2. Value of the tuning objective on the development set for each system after its own tuning.

4.4 Metrics

Metrics are computed per case, averaged per utterance, and then across utterances; confidence intervals are 95% bootstrap intervals over utterances. The clean reference is cut into blocks of 256 samples. A block is speech-active when its energy is within 40 dB of the loudest block; all others are noise-only.

- ΔSNR: output SNR minus input SNR against the clean speech (waveform fidelity).
- ΔsegSNR: the same on speech-active blocks, each clamped to −10 to 35 dB.
- NR: energy of the noisy signal over that of the output on noise-only blocks, in dB.
- SD: distortion of speech blocks, $10 \log_{10}(\sum(\hat{s} - s)^2 / \sum s^2)$; lower is better.
- ΔLSD: change in log-spectral distance on frames that touch speech.
- MNR: musical-noise ratio, the kurtosis of the noise-only power spectra at the output over that at the input. Values above one mean more isolated spectral peaks than in the input.
- ΔPESQ (wide-band, ITU-T P.862.2) and ΔSTOI from the external packages `pesq` (0.0.4) and `pystoi` (0.4.1), for B0 to B5 only.

These metrics have to be read together: a system that mutes everything scores well on NR and badly on the others.

4.5 Statistics

Differences between B5 and a comparison system are tested on per-utterance means with a paired Wilcoxon signed-rank test. The p-values within each family of comparisons are Holm-corrected. The utterance, not the individual case, is the unit of analysis, because cases built from the same utterance are not independent.

4.6 Real recordings

Two recordings (speech in fan noise, speech in corridor noise) have no clean reference, so SNR, PESQ, and STOI cannot be computed. For these the code measures what the system does to pauses and to speech. A pause block is a block whose energy does not exceed the 95th percentile of the block energies of the matching noise-only recording plus 1 dB; a speech block exceeds that floor by 10 dB. Reported are the noise reduction and residual level in pauses, the musical-noise ratio in pauses, the change of the speech-block level, and Welch power spectra of the pauses (computed with the project's own estimator).

5 Results

5.1 Overall comparison

Table 3 gives the main result and Table 4 the remaining metrics, both on the test set.

System	ΔSNR (dB)	ΔsegSNR (dB)	ΔPESQ
B1 spectral subtraction (G1)	5.20 [4.58, 5.76]	3.03 [2.42, 3.60]	0.24 [0.23, 0.26]
B2 scipy.signal.wiener	0.52 [0.32, 0.67]	−0.75 [−1.04, −0.53]	−0.04 [−0.07, −0.02]
B3 noisereduce (default)	−1.93 [−2.22, −1.67]	−2.46 [−2.80, −2.15]	−0.21 [−0.25, −0.16]
B4 Wiener, plain SNR	4.74 [4.19, 5.23]	2.97 [2.35, 3.53]	0.17 [0.16, 0.18]
B5 HushWave (full)	5.44 [4.81, 6.01]	3.54 [2.89, 4.14]	0.25 [0.23, 0.27]

Table 3. Test set, 24 utterances, mean and 95% interval over utterances.

System	NR (dB)	SD (dB)	ΔLSD (dB)	MNR	ΔSTOI
B1 spectral subtraction (G1)	9.5	−16.1	−2.89	2.16	0.006
B2 scipy.signal.wiener	1.4	n/a	−1.33	1.56	−0.011
B3 noisereduce (default)	21.3	n/a	4.89	5.04	−0.035
B4 Wiener, plain SNR	7.2	−19.0	−2.22	1.98	0.006
B5 HushWave (full)	10.0	−17.3	−2.85	2.56	0.003

Table 4. Test set, remaining metrics (means). NR: noise reduction, SD: speech distortion, MNR: musical-noise ratio. SD is not defined for B2 and B3.

HushWave (B5) has the highest SNR improvement, 5.44 dB, and the highest segmental SNR improvement, 3.54 dB. Its advantage over spectral subtraction is small: the median per-utterance difference is 0.26 dB in ΔSNR and 0.55 dB in ΔsegSNR, both significant after Holm correction. Against the plain-SNR Wiener gain (B4) the median difference is 0.75 dB, which measures what decision-directed estimation adds. Table 5 lists all paired tests.

Comparison	Metric	Median difference	Holm p	Significant
B5 vs B1	ΔSNR	0.264	<0.001	yes
B5 vs B1	ΔsegSNR	0.546	<0.001	yes
B5 vs B1	NR	0.410	<0.001	yes
B5 vs B1	ΔPESQ	0.007	0.415	no
B5 vs B1	ΔSTOI	−0.002	<0.001	yes
B5 vs B4	ΔSNR	0.749	<0.001	yes
B5 vs B4	ΔsegSNR	0.575	<0.001	yes
B5 vs B4	NR	2.650	<0.001	yes
B5 vs B4	ΔPESQ	0.080	<0.001	yes
B5 vs B4	ΔSTOI	−0.003	<0.001	yes

Table 5. Paired Wilcoxon tests of B5 against the two closest systems, on per-utterance means. Positive differences favour B5.

- **Perceptual quality.** B5 improves PESQ-WB by 0.25 (interval 0.25 [0.23, 0.27]), which equals spectral subtraction (0.24; Holm $p = 0.415$) and exceeds B4 by 0.08. The general-purpose denoiser B3 lowers PESQ by −0.21.
- **Intelligibility.** No gain-based method improves STOI: B1, B4, and B5 change it by about 0.01 or less. B5 is marginally below B1, a difference that is detectable statistically and negligible in practice. B3 lowers STOI by 0.035.
- **Noise reduction against speech preservation.** B3 removes the most noise (21.3 dB against 10.0 dB for B5) but at the price of the speech: its SNR improvement is negative, and on the real recordings it lowers the speech level by −9.8 dB and −11.3 dB.

5.2 Dependence on input SNR and noise type

Figure 3 and Table 6 show that B5 has the highest mean ΔSNR at every input SNR of the grid, from 7.4 dB at −5 dB down to 2.9 dB at 15 dB. The improvement shrinks as the input SNR rises because less noise remains to be removed and the risk of distorting speech grows.

Input SNR (dB)	B1	B2	B3	B4	B5
−5	7.37	1.00	6.59	6.00	7.43
0	6.47	0.96	2.71	5.54	6.55
5	5.59	0.88	−1.64	5.13	5.81
10	4.21	0.48	−6.26	4.22	4.56
15	2.33	−0.70	−11.06	2.79	2.86

Table 6. Mean ΔSNR (dB) by input SNR, test set.

By noise type (Table 7, Figure 4), the gain is largest for stationary and narrow-band noises (hum, hallway, white, pink, traffic) and small for babble and amplitude-modulated noise. Two cases are weak. On the transient noise B5 gives −0.2 dB, that is, no benefit: a minimum-based tracker sees a transient as signal, not as noise. On babble, spectral subtraction is better than B5 (1.4 dB against 1.1 dB); babble has the temporal and spectral structure of speech, so any method that separates speech and noise by statistics has little to work with.

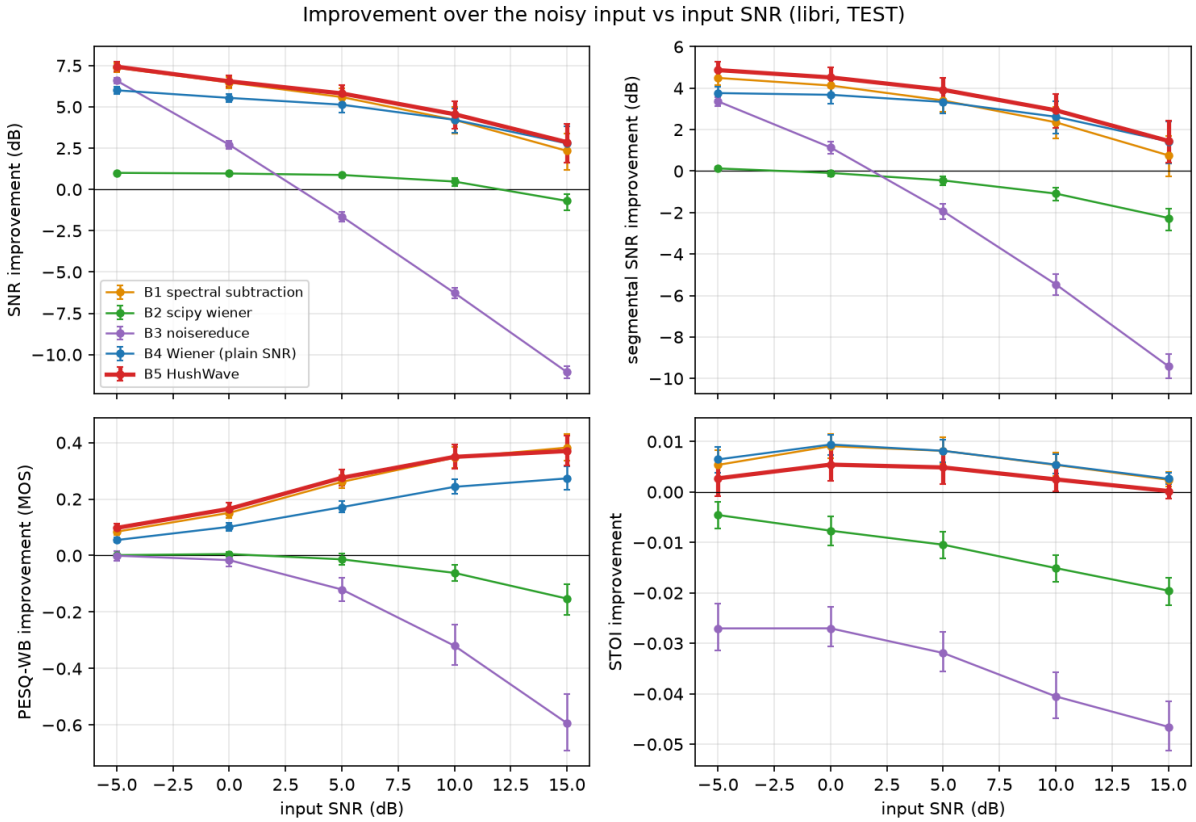


Figure 3. Improvement over the noisy input against input SNR, test set, with 95% intervals over utterances.

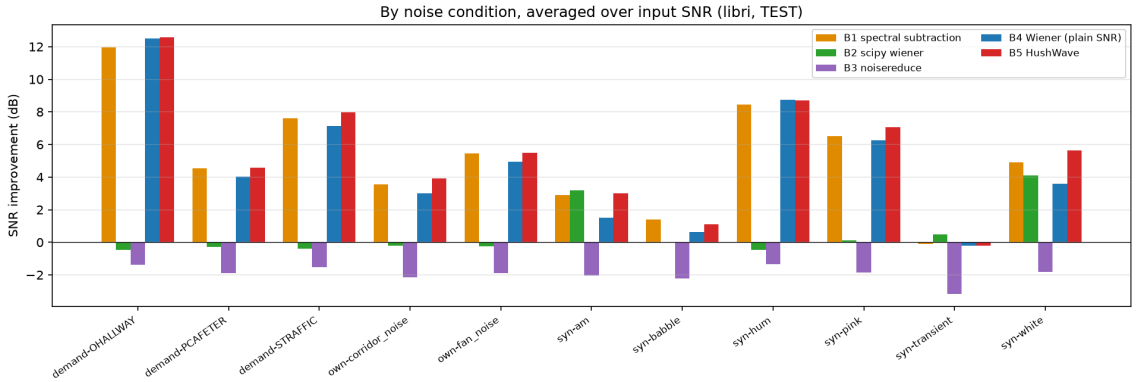


Figure 4. Mean ΔSNR by noise condition, averaged over input SNR.

Noise	B1	B4	B5	B5 NR (dB)	B5 ΔPESQ
demand-OHALLWAY	12.0	12.5	12.6	19.9	0.29
demand-PCAFETER	4.6	4.0	4.6	7.9	0.14
demand-STRAFFIC	7.6	7.1	8.0	14.0	0.37
own-corridor_noise	3.6	3.0	3.9	9.2	0.25
own-fan_noise	5.4	4.9	5.5	9.0	0.22
syn-am	2.9	1.5	3.0	5.2	0.08
syn-babble	1.4	0.6	1.1	2.7	0.01
syn-hum	8.4	8.7	8.7	18.5	0.77
syn-pink	6.5	6.2	7.1	13.2	0.39
syn-transient	−0.1	−0.2	−0.2	1.3	−0.01
syn-white	4.9	3.6	5.6	9.2	0.25

Table 7. Mean ΔSNR (dB) by noise condition, with the noise reduction and ΔPESQ of B5.

5.3 Ablations

Each ablation changes one component of B5 and re-evaluates on the test set with durations held constant (Table 8, Figure 5).

Variant	ΔSNR (dB)	ΔsegSNR (dB)	NR (dB)	Holm p
B5 (full system)	5.44 [4.81, 6.01]	3.54	10.0	
gain rule subtraction	4.62 [3.95, 5.28]	2.47	14.5	<0.001
noise tracker minstats	5.55 [4.90, 6.15]	3.56	11.7	0.011
no rise limit	5.55 [4.86, 6.22]	3.22	16.6	0.604
pre-filter none	3.77 [3.56, 3.98]	3.18	6.6	<0.001
pre-filter biquad	1.03 [0.42, 1.60]	−0.69	9.8	<0.001
weak decision-directed memory	4.98 [4.40, 5.49]	3.20	7.8	<0.001
lower gain floor	5.44 [4.81, 6.01]	3.54	10.1	1.000
higher gain floor	5.43 [4.81, 6.00]	3.50	9.7	0.003
frame length 16 ms	5.02 [4.41, 5.60]	3.19	11.5	<0.001
frame length 64 ms	5.48 [4.85, 6.04]	3.54	9.3	1.000

Table 8. Test set, B5 and ablations. The last column is the Holm-corrected p-value of the paired test against B5 on ΔSNR.

- **Pre-filter.** Replacing the zero-phase mask by the causal biquad costs 4.4 dB of SNR improvement, and removing the high-pass costs 1.7 dB. The two filters have matched magnitudes by design, so the difference is attributed to the phase.
- **Rule.** Spectral subtraction with the other components of B5 is worse than the Wiener gain on SNR and on segmental SNR. It removes more noise (Table 8) but has a higher musical-noise ratio and distorts the speech more.
- **Frame length.** 16 ms frames are worse than 32 ms; 64 ms frames are not distinguishable from 32 ms.
- **Floor.** Halving $g_{\min}$ or tripling it changes ΔSNR by 0.01 dB or less.
- **Decision-directed smoothing.** Reducing the memory constant from 0.9 to 0.5 is worse.

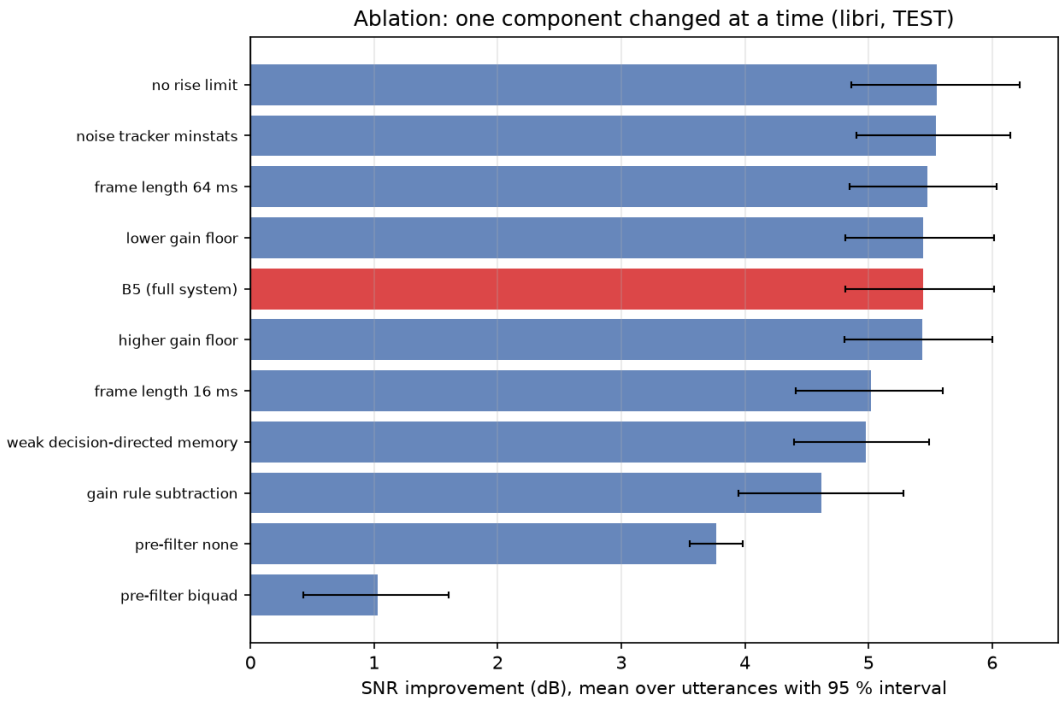


Figure 5. ΔSNR of B5 and its ablations with 95% intervals.

- **Tracker.** The minimum-statistics tracker scores about 0.1 dB *higher* than MCRA in ΔSNR (Holm $p = 0.011$), with similar segmental SNR. MCRA was chosen on development data. On the test data it has more noise reduction, but not a better SNR improvement. This is a selection that the test data does not support on the tuning objective.
- **Rise limit.** Without the rise limit the SNR improvement is 5.55 dB against 5.44 dB with it, which is not a significant difference (Holm $p = 0.604$). The limit adds about 0.3 dB of segmental SNR and *costs* noise reduction: 10.0 dB with the limit against 16.6 dB without it. The limit was chosen by the development objective, which does not see this cost.

5.4 Time-domain and frequency-domain analysis

Figure 6 shows one test case (a cafeteria noise at 5 dB input SNR) in the time domain. The output follows the clean envelope and the noise floor between words is lower, while the strongest speech peaks keep their amplitude. Figure 7 shows the same case as spectrograms: the broad-band noise floor in the input is suppressed in the pauses and between harmonics, and the harmonic structure of the voiced segments is preserved. Residual energy remains in short isolated cells in the pauses, which is what the musical-noise ratio measures, and weak high-frequency speech components are attenuated together with the noise.

Figure 8 shows the mean gain per frequency for the same case. The mean gain in speech-active blocks is clearly higher than in noise-only blocks over the whole band, and that difference is what separates speech from noise. Speech-active blocks are still attenuated on average, because many of their cells hold only noise. Both curves fall steeply below 100 Hz, where the high-pass acts.

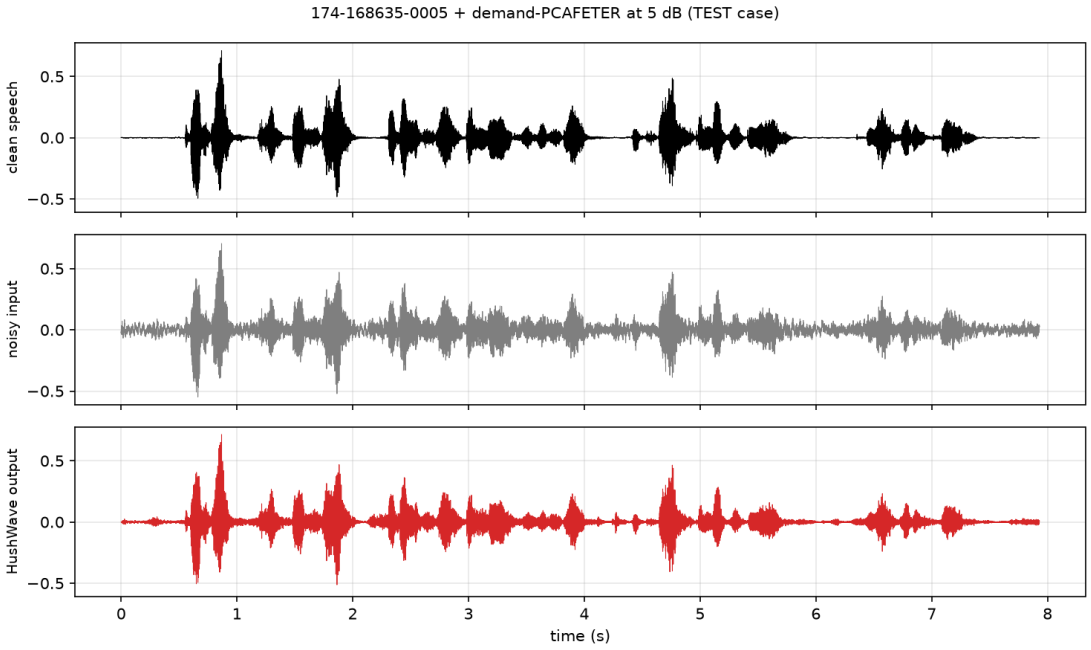


Figure 6. Clean speech, noisy input, and HushWave output for one test case.

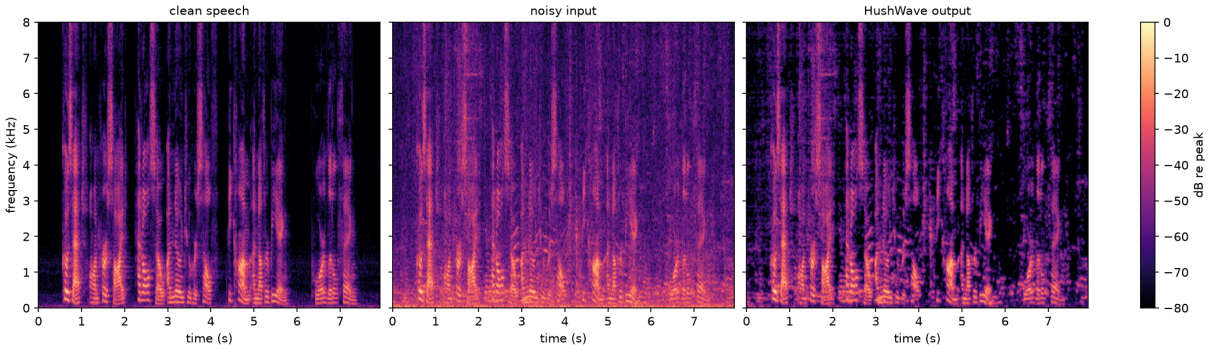


Figure 7. Spectrograms (dB relative to the peak) of the same case.

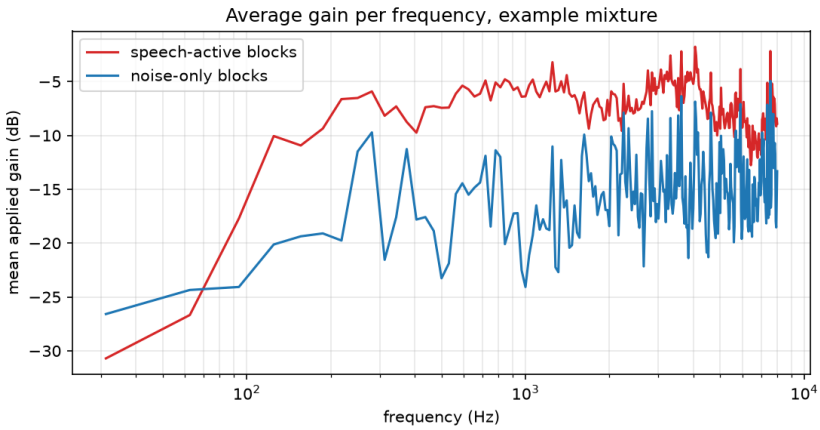


Figure 8. Mean gain applied by B5 per frequency, in speech-active and noise-only blocks.

5.5 Real recordings

Table 9 and Figure 9 show the result on the two recordings without a clean reference.

System	NR in pauses (dB)	Pause level (dBFS)	Speech level (dB)	MNR
Speech in fan noise				
B1 spectral subtraction (G1)	3.9	−46.1	−0.6	2.77
B2 scipy.signal.wiener	1.5	−43.6	−0.2	2.19
B3 noisereduce (default)	15.1	−57.3	−9.8	6.85
B4 Wiener, plain SNR	2.3	−44.5	−0.1	2.11
B5 HushWave (full)	2.8	−45.0	−0.2	2.87
Speech in corridor noise				
B1 spectral subtraction (G1)	2.1	−39.7	−0.3	1.48
B2 scipy.signal.wiener	0.7	−38.3	−0.2	0.99
B3 noisereduce (default)	20.1	−57.7	−11.3	6.39
B4 Wiener, plain SNR	1.8	−39.4	−0.1	1.33
B5 HushWave (full)	1.9	−39.5	−0.2	1.42

Table 9. Real recordings. NR: noise reduction in pause blocks; pause level relative to full scale; speech level: change of the energy of speech blocks.

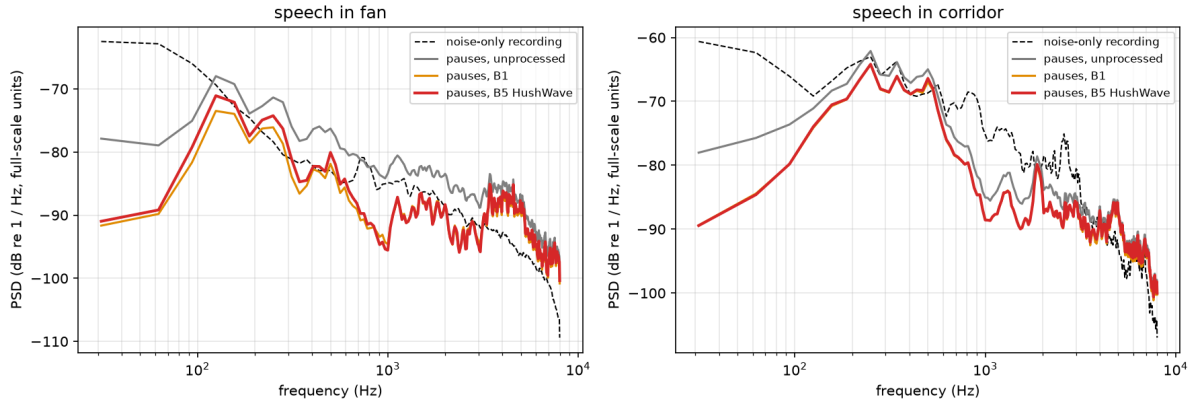


Figure 9. Welch power spectra of the pauses before and after processing, with the noise-only recording.

This is the weakest result of the project. HushWave reduces the noise in pauses by only 2.8 dB (fan) and 1.9 dB (corridor), while spectral subtraction reaches 3.9 dB and 2.1 dB. For comparison, mixtures of LibriSpeech with the same two noise recordings give a noise reduction of 9.0 dB and 9.2 dB for B5 (Table 7). The two measures are defined differently, but the gap suggests that the continuous speech of these recordings, with few and short pauses, limits the trackers, which then have few moments in which to observe noise alone. We did not isolate this. The spectra in Figure 9 show that the reduction is concentrated below 1 kHz, with little change above 3 kHz. We have not tested why; the candidates are low-level speech and breath in the pause blocks and the recording noise floor. The general-purpose denoiser B3 removes 15 to 20 dB of pause noise, but the speech level falls by about 10 to 11 dB, and its musical-noise ratio is the highest.

5.6 Own-voice corpus

On the 6 test segments of the own voice (Table 10) the ordering is the same as on LibriSpeech: B5 has a mean ΔSNR of 5.53 [4.34, 6.31] against 5.12 for B1. With six segments the intervals are

wide and a Holm-corrected test cannot reach significance, so this corpus supports the direction of the result and nothing more.

System	ΔSNR (dB)	ΔsegSNR (dB)
B1 spectral subtraction (G1)	5.12 [4.04, 5.83]	3.17
B2 scipy.signal.wiener	0.64 [0.54, 0.76]	−0.57
B3 noisereduce (default)	−2.39 [−2.56, −2.22]	−3.29
B4 Wiener, plain SNR	5.08 [4.33, 5.57]	3.51
B5 HushWave (full)	5.53 [4.34, 6.31]	3.84

Table 10. Own-voice test segments, mean and 95% interval.

5.7 Consistency between development and test

On the development set the tuning objective of B5 is 5.06 against 4.65 for B1. On the test set the same quantity is 4.49 against 4.11. The margin is stable, and there is no sign that the parameters overfit the development data.

5.8 Latency and real-time performance

The algorithmic delay is 31.9 ms (511 samples), constant for every block size. Table 11 and Figure 10 give the measured cost per block on the machine used for the measurements. For 256-sample (16 ms) blocks the real-time factor is 0.020 and the slowest block takes 0.56 ms against a budget of 16 ms. The peak memory of a ten-second stream is 102 kB. The engine therefore has a large margin on a laptop CPU; these figures describe that one machine and should not be generalised to an embedded target without measurement.

Block (samples)	Block (ms)	Real-time factor	Worst block (ms)	Budget (ms)
128	8.0	0.0218	0.43	8.0
256	16.0	0.0204	0.56	16.0
480	30.0	0.0193	0.76	30.0
1024	64.0	0.0191	1.41	64.0

Table 11. Cost of the streaming engine per block size.

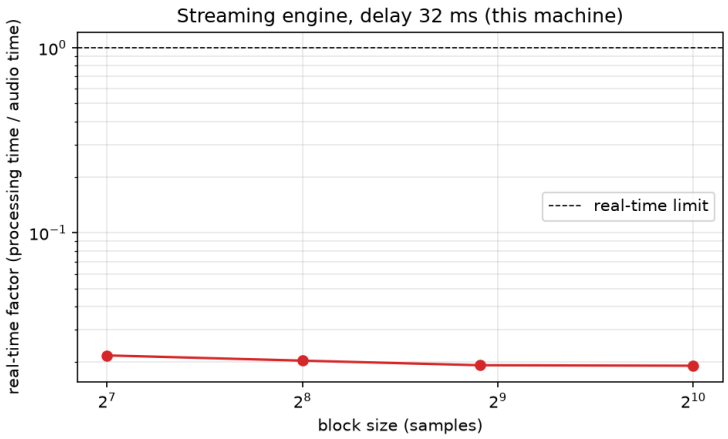


Figure 10. Real-time factor against block size.

6 Demonstration

The demonstration is a local web interface served by the project (`make demo-replay` and `make demo-live`). The browser acts only as the audio device. In the live mode it captures the microphone at 16 kHz with its own noise suppression, echo cancellation, and gain control switched off, sends blocks of 256 samples to the Python engine, and plays what returns. All signal processing runs in the same streaming engine that was evaluated, with the frozen tuned settings read from `results/tuning.json`. Nothing is retuned for the demonstration.

Replay. Two excerpts of the real recordings, each in three versions that share one playback level: the original, spectral subtraction, and HushWave. The versions can be switched during playback without a gap, and a spectrogram of the selected version moves with the playhead. The excerpts are cut from a continuous run over the whole recording, so they show the system in steady state. The measurements beside them are those of Table 9.

Live. The microphone passes through the engine. An enhancement switch cross-fades between the raw and the enhanced signal, input and output levels are displayed, and the interface reports the algorithmic delay, the processing time per block, the round trip to the engine, the output buffer, and dropouts. The estimated delay from microphone to headphones adds block, algorithmic delay, round trip, and buffer; it excludes the latency of the audio device. Headphones are required, because speakers feed back into the microphone.

The replay mode needs no microphone and no audio input permission, so the demonstration works even when the live path is unavailable.

7 Discussion and limitations

What the evidence supports. HushWave is the best of the compared systems on waveform measures at every input SNR, and the margin over a well-tuned spectral subtraction is real but small. It matches spectral subtraction on PESQ. It does not improve intelligibility as measured by STOI, and no method of this family does. The largest benefit of the work is not a new algorithm but a controlled comparison in which each component is isolated and tuned fairly.

Weaknesses.

1. The musical-noise ratio of B5 (2.56) is the highest of the classical systems (B1 2.16, B4 1.98). The tuning objective cannot see musical noise, and the development data switched gain smoothing off.
2. There is no improvement on transient noise, and spectral subtraction is better on babble.
3. On the real recordings the noise reduction in pauses is no better than that of spectral subtraction (Table 9).
4. The rise limit, selected on development data, gives no significant SNR gain on the test data and costs noise reduction. The MCRA tracker, also selected on development data, is slightly worse in ΔSNR than minimum statistics on the test data. Both are listed in the ablation and were not changed after the test evaluation.
5. The own-voice corpus has six segments, which is too few for corrected significance tests.

Threats to validity. Test mixtures are synthetic; real conditions add reverberation, non-stationary noise, and microphone characteristics that are absent here. The fan and corridor recordings differ in format (the corridor files are 48 kHz stereo, the others 44.1 kHz dual mono), which digital mixing makes harmless for the mixtures but which deviates from the recording protocol. The tuning objective is a waveform measure; a perceptual objective could select different parameters. PESQ and STOI come from external packages and were used as published. Timing was measured on one machine.

Possible next steps. A perceptual or musical-noise term in the tuning objective; a detector that treats transients as noise; longer recordings with more pauses for the real-recording evaluation; and a port of the engine to a fixed-point microcontroller.

8 Conclusion

A classical single-channel enhancer, implemented from first principles and evaluated under a protocol that prevents tuning on the test data, improves the SNR by 5.44 dB on average, with a small but statistically consistent advantage over spectral subtraction and without a perceptual gain over it. The streaming engine meets a real-time budget with a large margin and a delay of 31.9 ms. The results also show the limits of the approach: no intelligibility gain, no help with transient or babble noise, and modest noise reduction on real recordings with few pauses. These limits are reported as they are.

References

[1] S. F. Boll, “Suppression of acoustic noise in speech using spectral subtraction,” *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 27, no. 2, pp. 113–120, 1979.

[2] Y. Ephraim and D. Malah, “Speech enhancement using a minimum mean-square error short-time spectral amplitude estimator,” *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 32, no. 6, pp. 1109–1121, 1984.

[3] Y. Ephraim and D. Malah, “Speech enhancement using a minimum mean-square error log-spectral amplitude estimator,” *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 33, no. 2, pp. 443–445, 1985.

[4] R. Martin, “Noise power spectral density estimation based on optimal smoothing and minimum statistics,” *IEEE Trans. Speech Audio Process.*, vol. 9, no. 5, pp. 504–512, 2001.

[5] I. Cohen and B. Berdugo, "Noise estimation by minima controlled recursive averaging for robust speech enhancement," *IEEE Signal Process. Lett.*, vol. 9, no. 1, pp. 12–15, 2002.

[6] P. Scalart and J. V. Filho, "Speech enhancement based on a priori signal to noise estimation," in *Proc. IEEE ICASSP*, 1996, pp. 629–632.

[7] D. W. Griffin and J. S. Lim, "Signal estimation from modified short-time Fourier transform," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 32, no. 2, pp. 236–243, 1984.

[8] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, no. 90, pp. 297–301, 1965.

[9] P. D. Welch, "The use of fast Fourier transform for the estimation of power spectra," *IEEE Trans. Audio Electroacoust.*, vol. 15, no. 2, pp. 70–73, 1967.

[10] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "Librispeech: an ASR corpus based on public domain audio books," in *Proc. IEEE ICASSP*, 2015, pp. 5206–5210.

[11] J. Thiemann, N. Ito, and E. Vincent, "The diverse environments multi-channel acoustic noise database (DEMAND)," in *Proc. Meetings on Acoustics*, vol. 19, 2013.

[12] ITU-T Recommendation P.862.2, "Wideband extension to Recommendation P.862 for the assessment of wideband telephone networks and speech codecs," 2007.

[13] C. H. Taal, R. C. Hendriks, R. Heusdens, and J. Jensen, "An algorithm for intelligibility prediction of time-frequency weighted noisy speech," *IEEE Trans. Audio, Speech, Lang. Process.*, vol. 19, no. 7, pp. 2125–2136, 2011.

[14] Y. Uemura, Y. Takahashi, H. Saruwatari, K. Shikano, and K. Kondo, "Automatic optimization scheme of spectral subtraction based on musical noise assessment via higher-order statistics," in *Proc. IWAENC*, 2008.

[15] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[16] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, 1945.

A Reproducing the results

All commands run from the repository root in a Python 3.11 environment (the results were generated with Python 3.11.18, NumPy 2.4.6, SciPy 1.17.1, `pesq` 0.0.4, `pystoi` 0.4.1, `noisereduce` 3.0.3).

Command	Purpose
`make setup`	create the environment
`make data`	download LibriSpeech and DEMAND, verify checksums
`make verify`	lint, type check, tests with coverage, results schema, data manifest
`make experiments`	tune on development data, evaluate once on test data
`make figures`	draw the figures from the results
`make results-doc`	write `docs/RESULTS.md`
`make examples`	build the replay clips from the recordings
`make demo-replay`	start the replay interface
`make demo-live`	start the live microphone interface
`make report`	build this report

B Requirement traceability

Requirement	Where it is met	Code
One algorithm implemented by the students	Section 3	`src/hushwave/`
Mathematical formulation	Sections 2 and 3, `docs/THEORY.md`	`theory.py`
Time-domain analysis	Figure 6, Section 5.4	`figures.py`
Frequency-domain analysis	Figures 7, 8, 9	`figures.py`, `spectral.py`
Raw against processed, with numbers	Tables 3, 4, 9	`metrics.py`
Justified sampling frequency	Section 2.2	`theory.py`
Working real-time demonstration	Sections 5.8 and 6	`live.py`, `demo_server.py`